



IN242 - Kecerdasan Mesin

Chapter 9

Unsupervised Learning Techniques

Chapter 9: Unsupervised Learning Techniques, Clustering, K-Means

Unsupervised Learning Techniques

- Clustering

The goal is to group similar instances together into clusters. Clustering is a great tool for data analysis, customer segmentation, recommender systems, search engines, image segmentation, semi-supervised learning, dimensionality reduction, and more.

- Anomaly detection

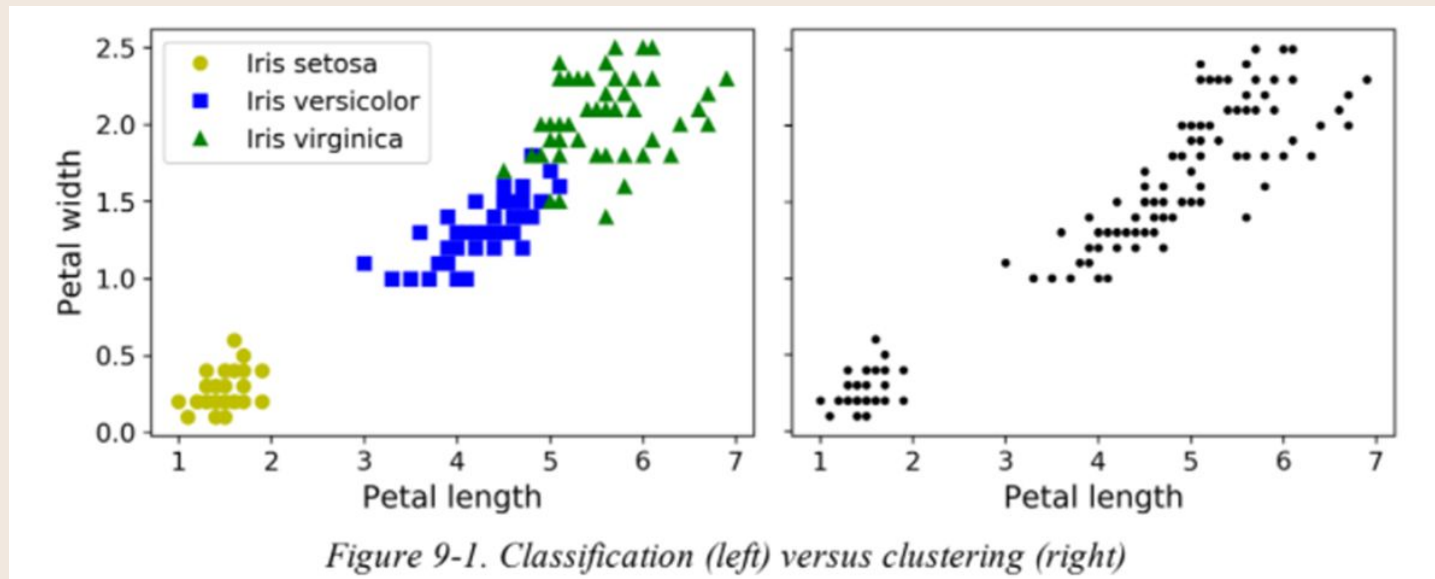
The objective is to learn what “normal” data looks like, and then use that to detect abnormal instances, such as defective items on a production line or a new trend in a time series.

- Density estimation

This is the task of estimating the probability density function (PDF) of the random process that generated the dataset. Density estimation is commonly used for anomaly detection: instances located in very low-density regions are likely to be anomalies. It is also useful for data analysis and visualization.

Clustering

The dataset has two additional features (sepal length and width), not represented here, and clustering algorithms can make good use of all features, so in fact they identify the three clusters fairly well (e.g., using a Gaussian mixture model, only 5 instances out of 150 are assigned to the wrong cluster).



Clustering is used in a wide variety of applications

- For customer segmentation

You can cluster your customers based on their purchases and their activity on your website. This is useful to understand who your customers are and what they need, so you can adapt your products and marketing campaigns to each segment.

- For data analysis

When you analyze a new dataset, it can be helpful to run a clustering algorithm, and then analyze each cluster separately.

- As a dimensionality reduction technique

Once a dataset has been clustered, it is usually possible to measure each instance's affinity with each cluster (affinity is any measure of how well an instance fits into a cluster). Each instance's feature vector x can then be replaced with the vector of its cluster affinities. If there are k clusters, then this vector is k -dimensional.

- For anomaly detection (also called outlier detection)

Any instance that has a low affinity to all the clusters is likely to be an anomaly. Anomaly detection is particularly useful in detecting defects in manufacturing, or for fraud detection.

- For semi-supervised learning

If you only have a few labels, you could perform clustering and propagate the labels to all the instances in the same cluster. This technique can greatly increase the number of labels available for a subsequent supervised learning algorithm, and thus improve its performance.

- For search engines

when a user provides a reference image, all you need to do is use the trained clustering model to find this image's cluster, and you can then simply return all the images from this cluster.

- To segment an image

Image segmentation is used in many object detection and tracking systems, as it makes it easier to detect the contour of each object.

K-Means

Consider the unlabeled dataset represented in Figure 9-2: you can clearly see five blobs of instances. The K-Means algorithm is a simple algorithm capable of clustering this kind of dataset very quickly and efficiently, often in just a few iterations.

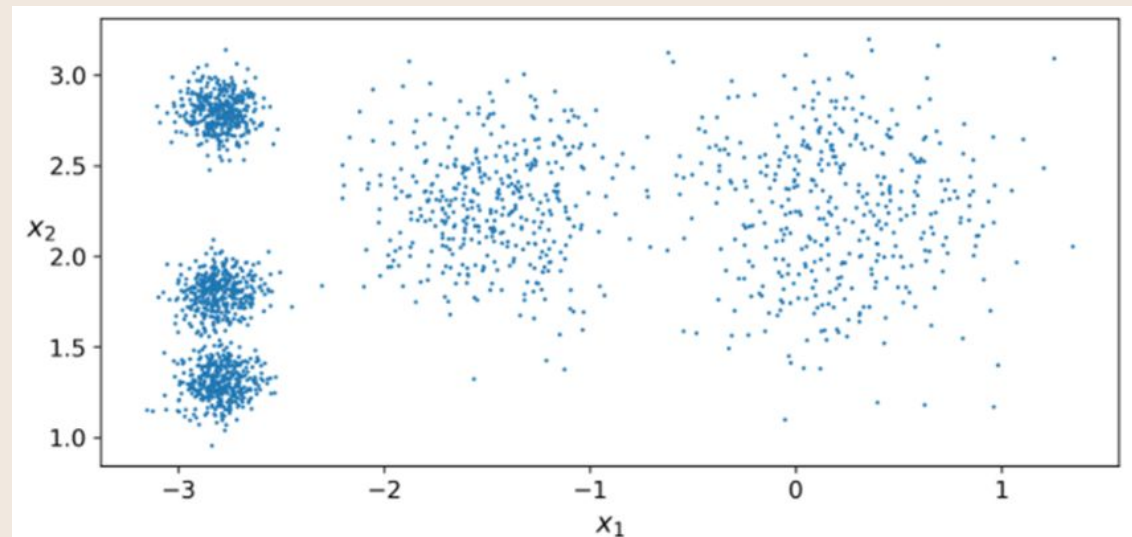


Figure 9-2. An unlabeled dataset composed of five blobs of instances

K-Means clusterer on this dataset. It will try to find each blob's center and assign each instance to the closest blob:

```
from sklearn.cluster import KMeans  
k = 5  
kmeans = KMeans(n_clusters=k)  
y_pred = kmeans.fit_predict(X)
```

Note that you have to specify the number of clusters k that the algorithm must find. In this example, it is pretty obvious from looking at the data that k should be set to 5.

In the context of clustering, an instance's label is the index of the cluster that this instance gets assigned to by the algorithm.

The KMeans instance preserves a copy of the labels of the instances it was trained on, available via the `labels_` instance variable:

```
>>> y_pred  
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)  
>>> y_pred is kmeans.labels_  
True
```

We can also take a look at the five centroids that the algorithm found:

```
>>> kmeans.cluster_centers_  
array([[ -2.80389616,  1.80117999],  
       [  0.20876306,  2.25551336],  
       [ -2.79290307,  2.79641063],  
       [ -1.46679593,  2.28585348],  
       [ -2.80037642,  1.30082566]])
```

can easily assign new instances to the cluster whose centroid is closest:

```
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])  
>>> kmeans.predict(X_new)  
array([1, 1, 2, 2], dtype=int32)
```

If you plot the cluster's decision boundaries, you get a Voronoi tessellation (see Figure 9-3, where each centroid is represented with an X).

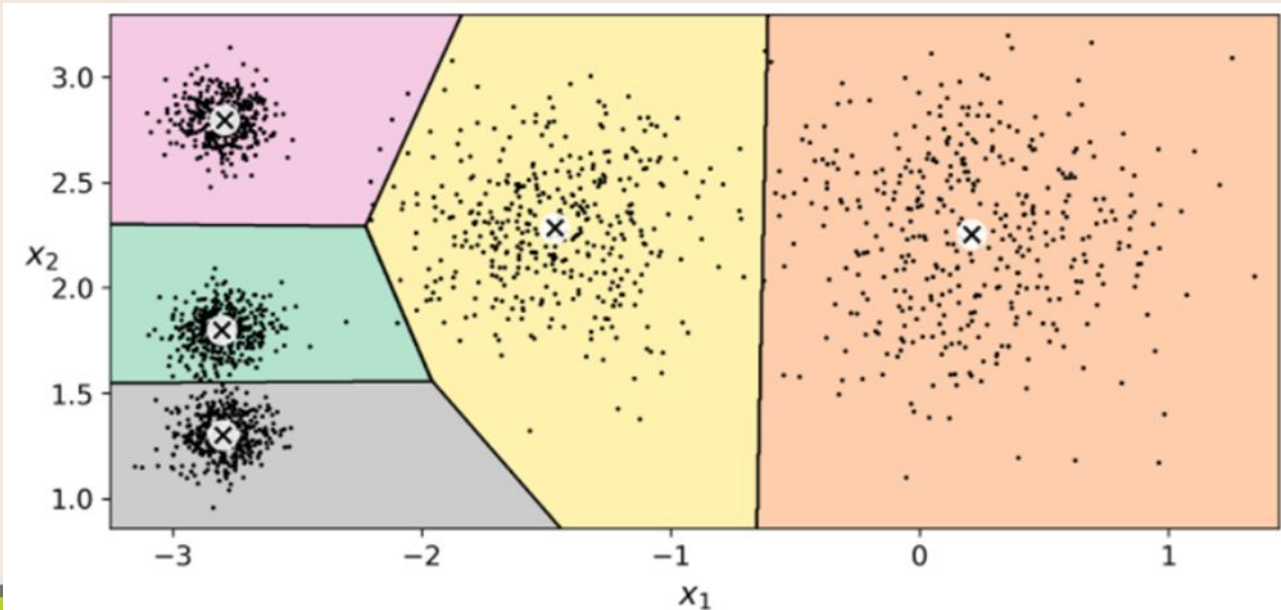


Figure 9-3. K-Means decision boundaries (Voronoi tessellation)

In the KMeans class, the transform() method measures the distance from each instance to every centroid:

```
>>> kmeans.transform(X_new)
array([[2.81093633, 0.32995317, 2.9042344 , 1.49439034, 2.88633901],
       [5.80730058, 2.80290755, 5.84739223, 4.4759332 , 5.84236351],
       [1.21475352, 3.29399768, 0.29040966, 1.69136631, 1.71086031],
       [0.72581411, 3.21806371, 0.36159148, 1.54808703, 1.21567622]])
```

In this example, the first instance in X_new is located at a distance of 2.81 from the first centroid, 0.33 from the second centroid, 2.90 from the third centroid, 1.49 from the fourth centroid, and 2.89 from the fifth centroid.

The K-Means algorithm

You could easily label all the instances in the dataset by assigning each of them to the cluster whose centroid is closest.

You can see the algorithm in action

in Figure 9-4: the centroids are initialized randomly (top left), then the instances are labeled (top right), then the centroids are updated (center left), the instances are relabeled (center right), and so on. As you can see, in just three iterations, the algorithm has reached a clustering that seems close to optimal.

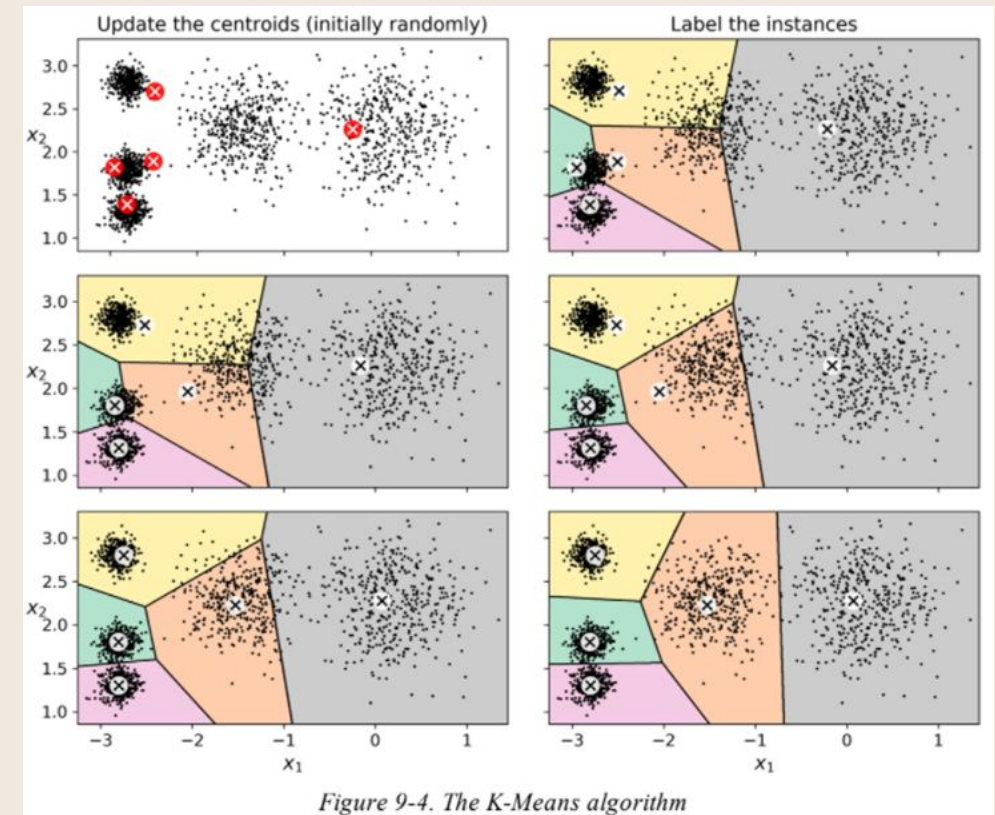
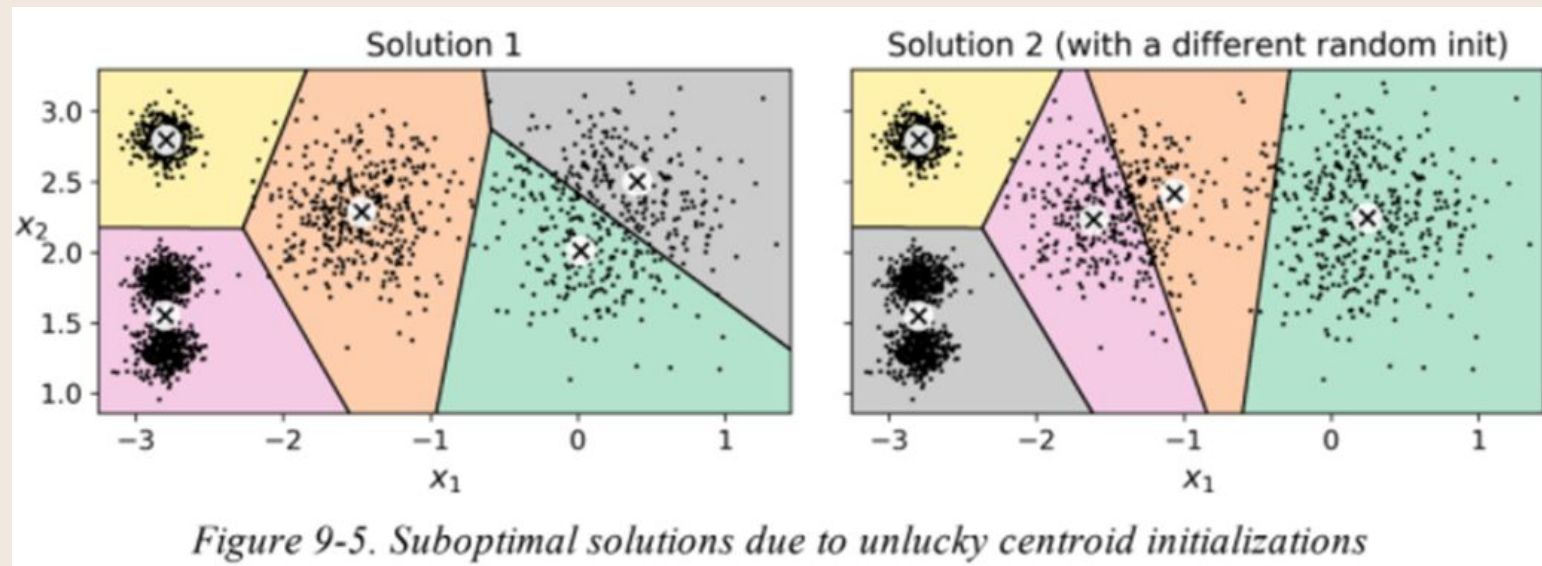


Figure 9-5 shows two suboptimal solutions that the algorithm can converge to if you are not lucky with the random initialization step.



Centroid initialization methods

If you happen to know approximately where the centroids should be, then you can set the `init` hyperparameter to a NumPy array containing the list of centroids, and set `n_init` to 1:

```
good_init = np.array([[-3, 3], [-3, 2], [-3, 1], [-1, 2], [0, 2]])  
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
```

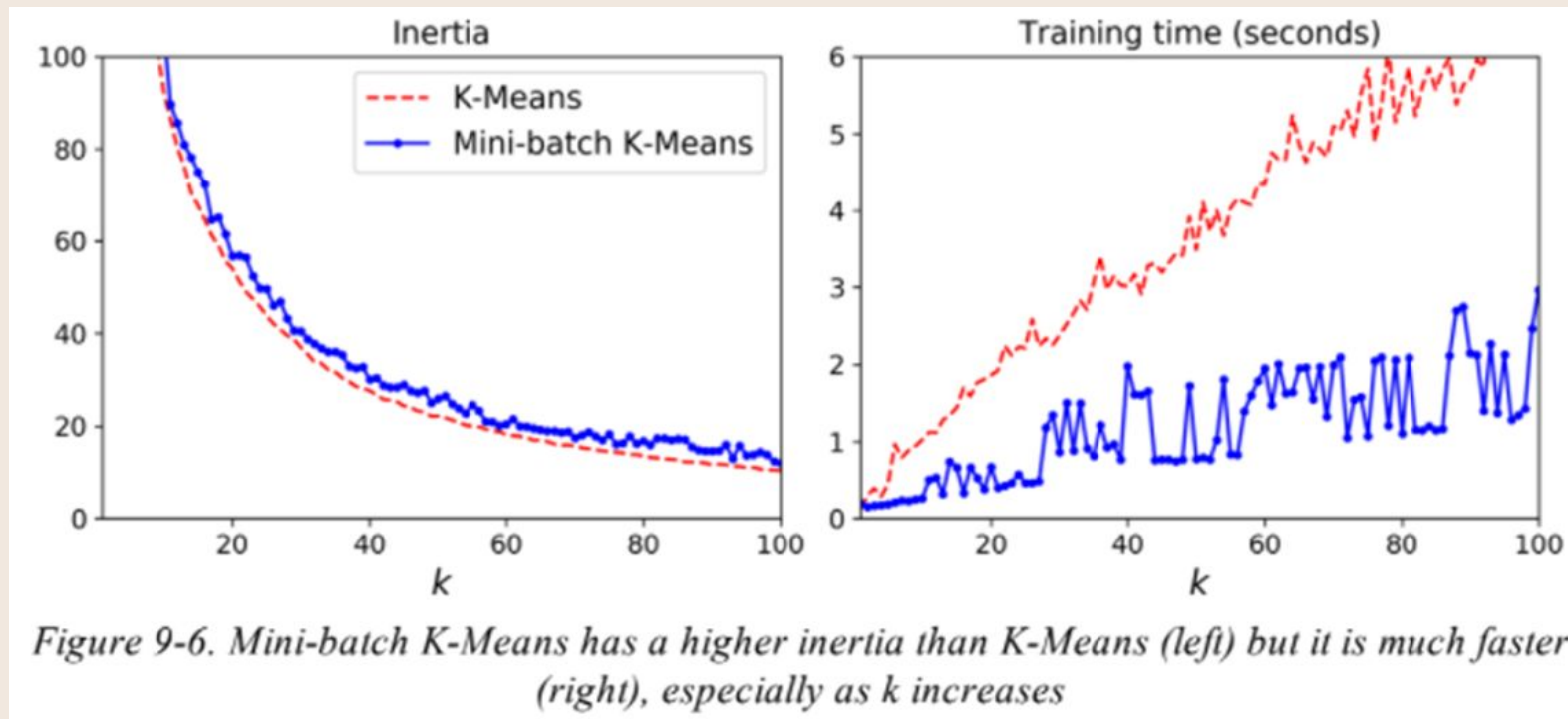
Accelerated K-Means and mini-batch K-Means

This is the algorithm the KMeans class uses by default (you can force it to use the original algorithm by setting the algorithm hyperparameter to "full", although you probably will never need to). Scikit-Learn implements this algorithm in the MiniBatchKMeans class. You can just use this class like the Kmeans class:

```
from sklearn.cluster import MiniBatchKMeans

minibatch_kmeans = MiniBatchKMeans(n_clusters=5)
minibatch_kmeans.fit(X)
```

The difference between the two curves remains fairly constant, but this difference becomes more and more significant as k increases, since the inertia becomes smaller and smaller. In the plot on the right, you can see that Mini-batch K-Means is much faster than regular K-Means, and this difference increases with k .



Finding the optimal number of clusters

So far, we have set the number of clusters k to 5 because it was obvious by looking at the data that this was the correct number of clusters. But in general, it will not be so easy to know how to set k , and the result might be quite bad if you set it to the wrong value. As you can see in Figure 9-7, setting k to 3 or 8 results in fairly bad models.

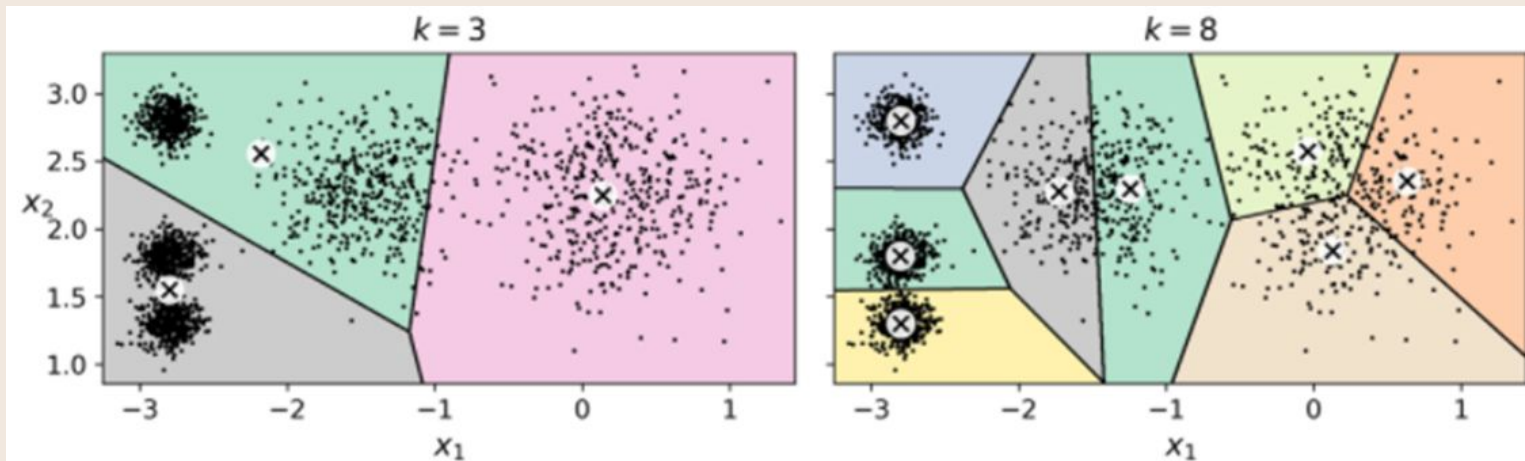
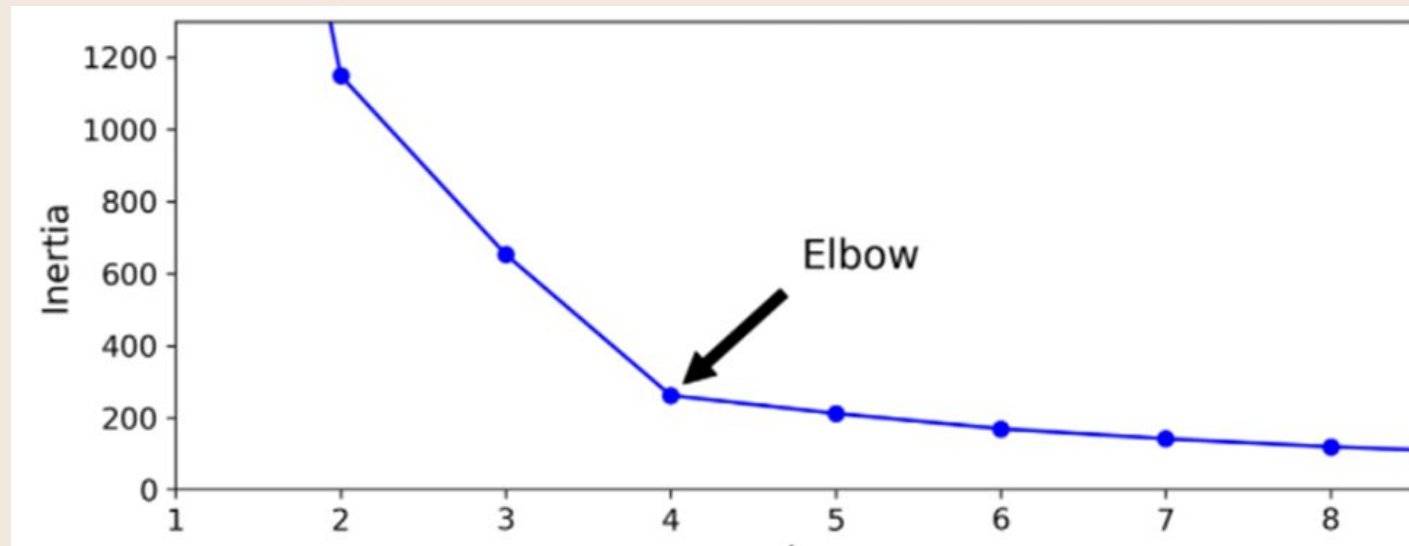


Figure 9-7. Bad choices for the number of clusters: when k is too small, separate clusters get merged (left), and when k is too large, some clusters get chopped into multiple pieces (right)

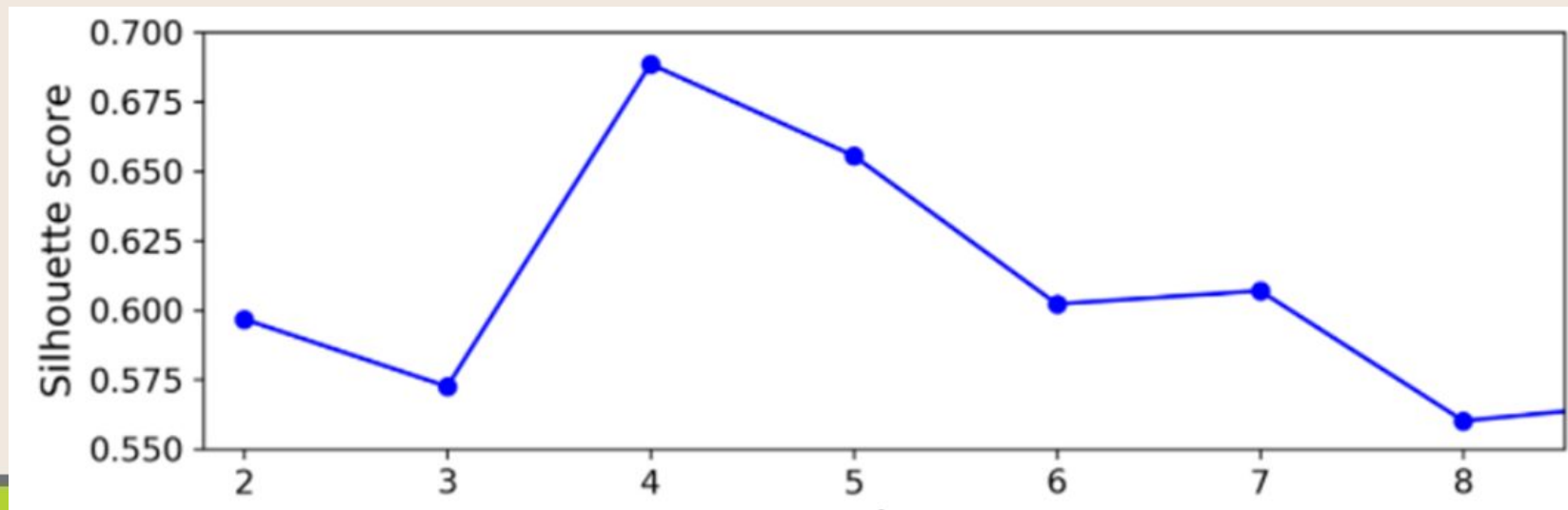
The inertia for $k=3$ is 653.2, which is much higher than for $k=5$ (which was 211.6). But with $k=8$, the inertia is just 119.1. Let's plot the inertia as a function of k .



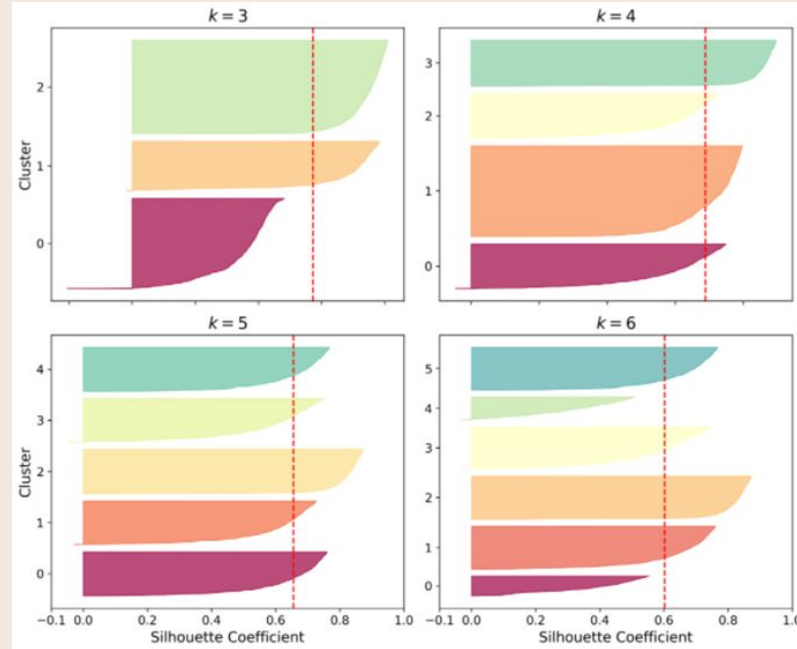
To compute the silhouette score, you can use Scikit-Learn's `silhouette_score()` function, giving it all the instances in the dataset and the labels they were assigned:

```
>>> from sklearn.metrics import silhouette_score  
>>> silhouette_score(X, kmeans.labels_)  
0.655517642572828
```

compare the silhouette scores for different numbers of clusters



This visualization is much richer than the previous one: although it confirms that $k = 4$ is a very good choice, it also underlines the fact that $k = 5$ is quite good as well, and much better than $k = 6$ or 7. More informative visualization is obtained when you plot every instance's silhouette coefficient, sorted by the cluster they are assigned to and by the value of the coefficient. This is called a silhouette diagram.



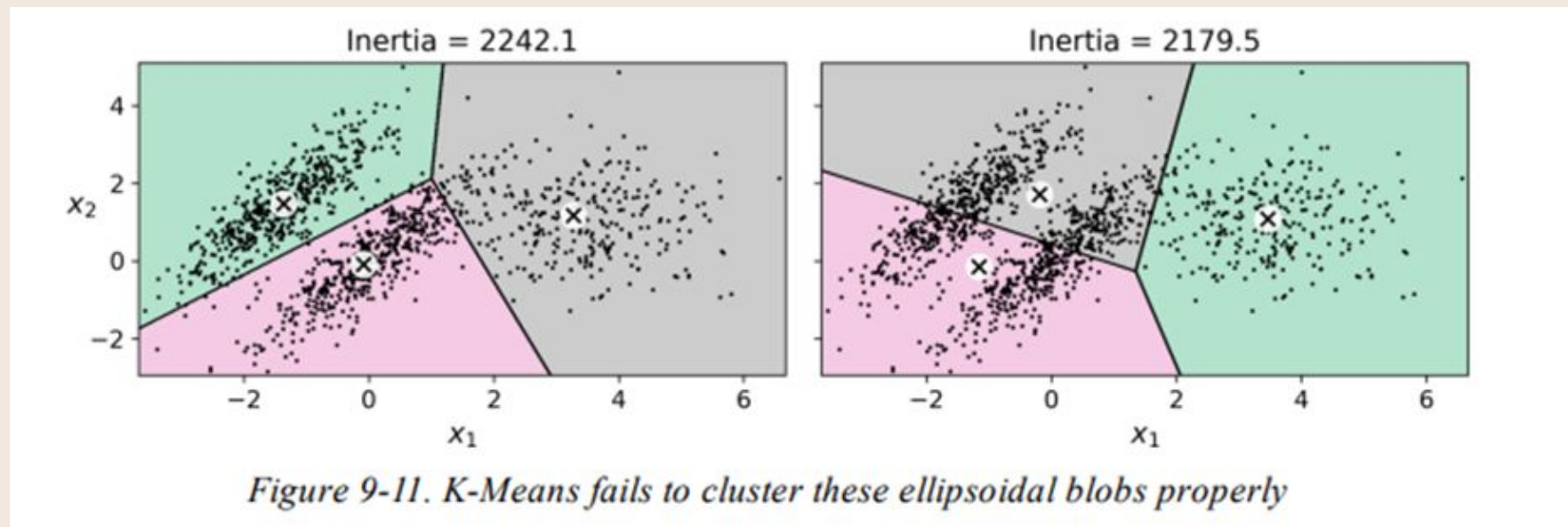
Chapter 9: Limits of K-Means, Using Clustering for Image Segmentation, Using Clustering for Preprocessing, Using Clustering for Semi-Supervised Learning

Limits of K-Means

Terlepas dari banyak kelebihanannya, terutama karena cepat dan skalabel, K-Means pun tidak sempurna.

Perlu menjalankan algoritma beberapa kali untuk menghindari solusi sub optimal, ditambah perlu menentukan jumlah cluster, yang bisa sangat merepotkan. Apalagi K-Means tidak berperilaku sangat baik ketika cluster memiliki ukuran yang bervariasi, kepadatan yang berbeda, atau bentuk nonspherical.

Misalnya pada Gambar 9-11 : Bagaimana K-Means mengelompokkan kumpulan data yang berisi tiga kluster elipsoidal yang berbeda dimensi, densitas, dan orientasi.



- Seperti yang terlihat, tak satu pun dari solusi ini ada gunanya.
- Solusi pada kiri lebih baik, tetapi masih memotong 25% dari cluster tengah dan menetapkannya ke cluster di sebelah kanan.
- Solusi di sebelah kanan sangat buruk, bahkan meskipun inersianya lebih rendah.
- Pada jenis cluster elips ini, Model campuran Gaussian bekerja dengan baik.

USING CLUSTERING FOR IMAGE SEGMENTATION

Segmentasi gambar adalah tugas membagi gambar menjadi beberapa segmen. Dalam segmentasi semantik, semua piksel yang merupakan bagian yang sama jenis objek ditugaskan ke segmen yang sama

Contoh :

Misalnya, dalam sistem penglihatan mobil yang mengemudi sendiri, semua piksel yang merupakan bagian dari gambar pejalan kaki mungkin ditugaskan ke segmen "pejalan kaki" (akan ada satu segmen yang berisi semua pejalan kaki).

Dalam segmentasi instance, semua piksel yang merupakan bagian dari objek individu yang sama ditugaskan ke segmen yang sama.

Dalam hal ini akan ada segmen yang berbeda untuk masing-masing pejalan kaki. Keadaan seni dalam segmentasi semantik atau instance saat ini dicapai dengan menggunakan arsitektur kompleks berdasarkan convolutional neural jaringan

1. Pertama, gunakan fungsi imread() Matplotlib untuk memuat gambar.

```
>>> from matplotlib.image import imread # or `from imageio import  
imread`  
>>> image =  
imread(os.path.join("images", "unsupervised_learning", "ladybug.png"))  
  
>>> image.shape  
(533, 800, 3)
```

Gambar direpresentasikan sebagai array 3D.

- Ukuran dimensi pertama adalah tinggi;
- kedua adalah lebarnya;
- ketiga adalah jumlah warna saluran, dalam hal ini merah, hijau, dan biru (RGB).

Dengan kata lain, untuk setiap pixel ada vektor 3D yang berisi intensitas warna merah, hijau, dan biru, masing-masing antara 0,0 dan 1,0 (atau antara 0 dan 255, jika Anda menggunakan imageio.imread()).

Beberapa gambar mungkin memiliki lebih sedikit saluran (channel), seperti gambar skala abu-abu (satu saluran). Dan beberapa gambar mungkin memiliki lebih banyak saluran, seperti gambar dengan saluran alfa tambahan untuk transparansi atau citra satelit, yang sering kali berisi saluran untuk banyak frekuensi cahaya (misalnya, inframerah).

2. Mengelompokkan dengan menggunakan K-Means

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```

- Misalnya, mengidentifikasi cluster warna untuk semua nuansa hijau. Berikutnya, untuk setiap warna (misalnya, hijau tua), ia mencari warna rata-rata piksel klaster warna.
- Misalnya, semua nuansa hijau dapat diganti dengan warna hijau muda yang sama (dengan asumsi warna rata-rata cluster hijau adalah hijau muda).
- Akhirnya, hal itu membentuk kembali daftar panjang warna ini untuk mendapatkan yang sama bentuk seperti gambar aslinya.



Figure 9-12. Image segmentation using K-Means with various numbers of color clusters

Kapan menggunakan kurang dari delapan kelompok, perhatikan bahwa warna merah mencolok kepik itu gagal mendapatkan klusternya sendiri: ia digabung dengan warna dari lingkungan. Ini karena K-Means lebih menyukai cluster dengan ukuran yang sama. Kepik itu kecil (jauh lebih kecil dari gambar lainnya) meskipun warnanya mencolok, K-Means gagal mendedikasikan sebuah cluster.

USING CLUSTERING FOR PREPROCESSING

- Pengelompokan dapat menjadi pendekatan yang efisien untuk pengurangan dimensi, sebagai langkah preprocessing sebelum algoritma pembelajaran yang diawasi.
- Sebagai contoh penggunaan pengelompokan untuk pengurangan dimensi.

Menangani kumpulan data digit, yang merupakan kumpulan data sederhana seperti MNIST yang berisi 1.797 gambar skala abu-abu 8×8 yang mewakili angka 0 hingga 9. Pertama, muat himpunan data:

```
from sklearn.datasets import load_digits  
  
X_digits, y_digits = load_digits(return_X_y=True)
```

Now, split it into a training set and a test set:

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits)
```

Next, fit a Logistic Regression model:

```
from sklearn.linear_model import LogisticRegression
```

```
log_reg = LogisticRegression()  
log_reg.fit(X_train, y_train)
```

Let's evaluate its accuracy on the test set:

```
>>> log_reg.score(X_test, y_test)  
0.9688888888888889
```

Akurasi 96,9%. Kemudian coba gunakan K-Means sebagai langkah preprocessing.

Membuat saluran yang akan membuat cluster pertama set pelatihan menjadi 50 cluster dan ganti gambar dengan jarak mereka ke 50 cluster ini, lalu terapkan Regresi Logistik model:

```
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("kmeans", KMeans(n_clusters=50)),
    ("log_reg", LogisticRegression()),
])
pipeline.fit(X_train, y_train)
```

WARNING

Since there are 10 different digits, it is tempting to set the number of clusters to 10. However, each digit can be written several different ways, so it is preferable to use a larger number of clusters, such as 50.

Now let's evaluate this classification pipeline:

```
>>> pipeline.score(X_test, y_test)  
0.9777777777777777
```

Telah terjadi pengurangi tingkat kesalahan hampir 30% (dari sekitar 3,1% hingga sekitar 2,2%)!

- Tetapi pemilihan jumlah cluster k secara sembarang; kita pasti bisa lebih baik. Karena K-Means hanyalah langkah pra-pemrosesan dalam pipa klasifikasi, menemukan nilai yang baik untuk k jauh lebih sederhana daripada sebelumnya. Tidak perlu melakukan analisis siluet atau meminimalkan inersia;
- nilai k terbaik adalah hanya salah satu yang menghasilkan kinerja klasifikasi terbaik selama validasi silang.

Kita dapat menggunakan GridSearchCV untuk menemukan jumlah optimal cluster:

```
from sklearn.model_selection import GridSearchCV

param_grid = dict(kmeans__n_clusters=range(2, 100))
grid_clf = GridSearchCV(pipeline, param_grid, cv=3, verbose=2)
grid_clf.fit(X_train, y_train)
```

Let's look at the best value for k and the performance of the resulting pipeline:

```
>>> grid_clf.best_params_
{'kmeans__n_clusters': 99}
>>> grid_clf.score(X_test, y_test)
0.9822222222222222
```

Dengan $k = 99$ cluster, kita dapatkan peningkatan akurasi yang signifikan, mencapai 98,22% akurasi pada set tes.

USING CLUSTERING FOR SEMI-SUPERVISED LEARNING

Kegunaan lain untuk pengelompokan adalah dalam pembelajaran semi-supervised, ketika kita memiliki banyak instance tidak berlabel dan sangat sedikit instance berlabel.

Melatih model Regresi Logistik pada sampel 50 instans berlabel dari kumpulan data digit:

```
n_labeled = 50
log_reg = LogisticRegression()
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

What is the performance of this model on the test set?

```
>>> log_reg.score(X_test, y_test)
0.8333333333333334
```

Akurasinya hanya 83,3%.

Pertama, kelompokkan set pelatihan menjadi 50 cluster. Kemudian untuk setiap cluster, kita cari gambar yang paling dekat dengan centroid. Kami akan menyebut gambar-gambar ini sebagai gambar representatif:

```
k = 50
kmeans = KMeans(n_clusters=k)
X_digits_dist = kmeans.fit_transform(X_train)
representative_digit_idx = np.argmin(X_digits_dist, axis=0)
X_representative_digits = X_train[representative_digit_idx]
```

Figure 9-13 shows these 50 representative images.

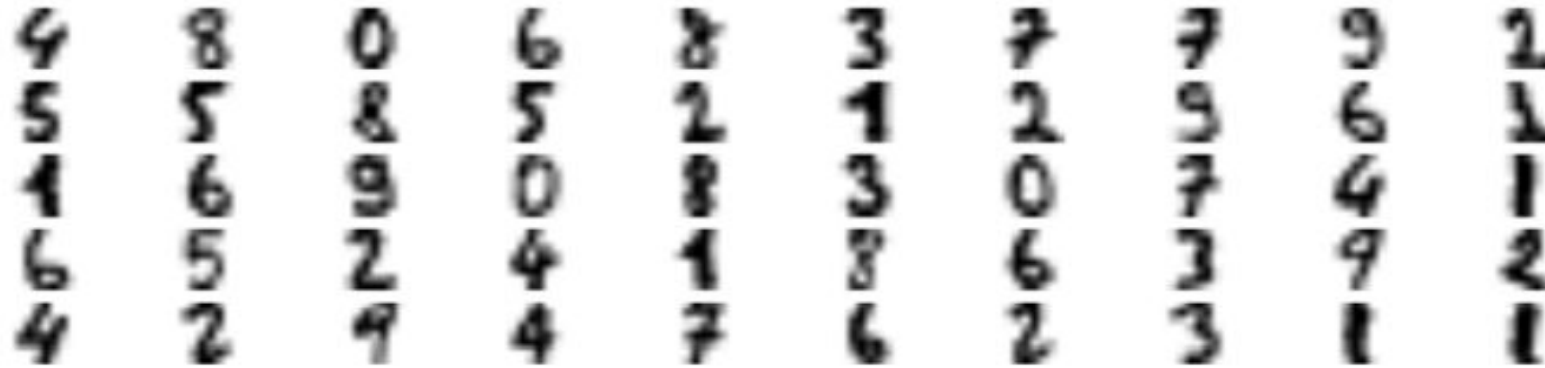


Figure 9-13. Fifty representative digit images (one per cluster)

Let's look at each image and manually label it:

```
y_representative_digits = np.array([4, 8, 0, 6, 8, 3, ..., 7, 6, 2, 3, 1,  
1])
```

Sekarang kami memiliki kumpulan data dengan hanya 50 instance berlabel, tetapi contoh acak, masing-masing adalah gambar representatif dari clusternya.

Let's make it better !

```
>>> log_reg = LogisticRegression()  
>>> log_reg.fit(X_representative_digits, y_representative_digits)  
>>> log_reg.score(X_test, y_test)  
0.9222222222222223
```

Kita melompat dari akurasi 83,3% menjadi 92,2%, meskipun hanya melatih model pada 50 instansi. Karena seringkali mahal dan menyulitkan untuk memberi label pada instance, terutama jika harus dilakukan secara manual oleh para ahli.

Lalu bagaimana jika kita menyebarkan label ke semua instance lain di cluster yang sama? Ini disebut label perambatan :

```
y_train_propagated = np.empty(len(X_train), dtype=np.int32)
for i in range(k):
    y_train_propagated[kmeans.labels_==i] = y_representative_digits[i]
```


Now let's train the model again and look at its performance:

```
>>> log_reg = LogisticRegression()  
>>> log_reg.fit(X_train, y_train_propagated)  
>>> log_reg.score(X_test, y_test)  
0.9333333333333333
```

Kita mendapat peningkatan akurasi yang wajar, tetapi tidak ada yang benar-benar mencengangkan. Masalahnya adalah kita **menyebarkan label setiap contoh perwakilan ke semua instance dalam cluster yang sama, termasuk instance yang terletak dekat ke batas-batas cluster, yang lebih mungkin untuk disalahartikan**

jika kita hanya menyebarkan label ke 20% instance yang paling dekat dengan centroid:

```
percentile_closest = 20

X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1

partially_propagated = (X_cluster_dist != -1)

X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train_propagated[partially_propagated]
```

Now let's train the model again on this partially propagated dataset:

```
>>> log_reg = LogisticRegression()  
>>> log_reg.fit(X_train_partially_propagated,  
y_train_partially_propagated)  
>>> log_reg.score(X_test, y_test)  
0.94
```

Dengan hanya 50 instance berlabel (hanya 5 contoh per kelas aktif rata-rata!), kami mendapat akurasi 94,0%, yang cukup dekat dengan kinerjanya Regresi Logistik pada dataset digit berlabel lengkap (96,9%).

Performa yang baik ini disebabkan oleh fakta bahwa yang disembarkan label sebenarnya cukup bagus—akurasinya mendekati 99%, karena kode berikut menunjukkan:

```
>>> np.mean(y_train_partially_propagated ==  
y_train[partially_propagated])  
0.9896907216494846
```

Chapter 9: DBSCAN, Other Clustering Algorithms

DBSCAN

Algoritma ini mendefinisikan cluster sebagai daerah tiada henti dengan kepadatan tinggi. Berikut adalah cara kerjanya:

- Untuk setiap instance, algoritma menghitung berapa banyak instance terletak dalam jarak kecil (epsilon) dari itu. Wilayah ini adalah disebut instance's ϵ -neighborhood.
- Jika sebuah instance memiliki setidaknya instance min_samples di lingkungannya (termasuk dirinya sendiri), maka itu dianggap sebagai inti contoh. Dengan kata lain, instance inti adalah yang berlokasi di daerah padat.
- Semua instance di sekitar instance inti dimiliki oleh klaster yang sama. Lingkungan ini mungkin termasuk instance inti lainnya oleh karena itu, urutan panjang dari sekitar instance inti membentuk satu cluster.
- Instance apa pun yang bukan instance inti dan tidak memilikinya instance inti dilingkungannya dianggap anomali.

Algoritma ini bekerja dengan baik jika semua cluster cukup padat dan jika mereka dipisahkan dengan baik oleh daerah dengan kepadatan rendah.

Kelas DBSCAN di ScikitLearn mudah digunakan. Mari kita coba di dataset moons diperkenalkan di Bab 5:

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.05)
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

Label dari semua instance sekarang tersedia di labels_ instance variabel:

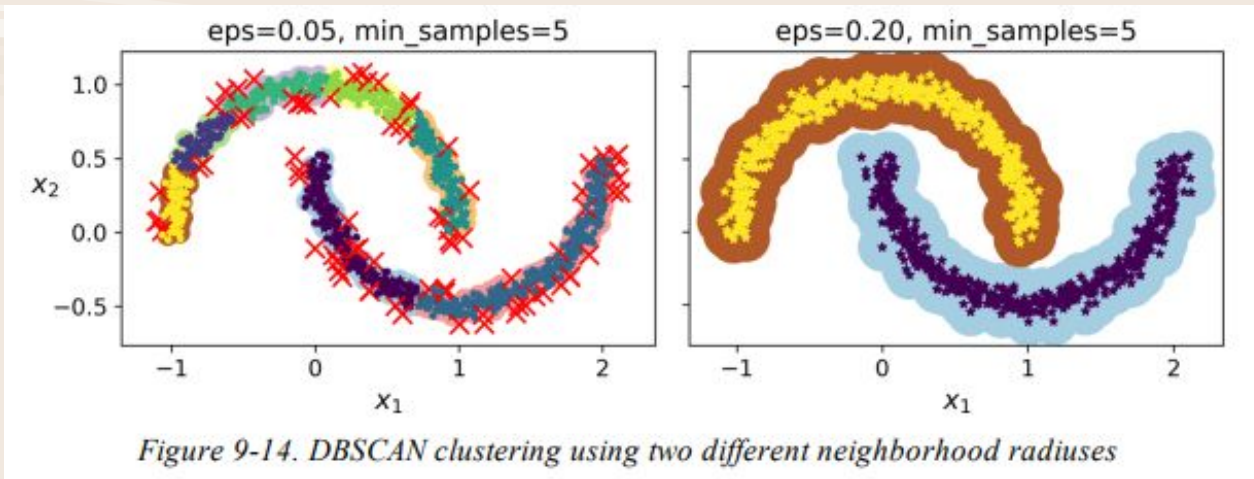
```
>>> dbscan.labels_
array([ 0,  2, -1, -1,  1,  0,  0,  0, ...,  3,  2,  3,  3,  4,  2,  6,
        3])
```

Perhatikan bahwa beberapa instance memiliki indeks cluster sama dengan -1, yang berarti bahwa mereka dianggap sebagai anomali oleh algoritma.

Indeks dari instance inti tersedia di `core_sample_indices_variable` dan instance inti itu sendiri tersedia di `components_instance variable`:

```
>>> len(dbscan.core_sample_indices_)
808
>>> dbscan.core_sample_indices_
array([ 0,  4,  5,  6,  7,  8, 10, 11, ..., 992, 993, 995, 997, 998,
       999])
>>> dbscan.components_
array([[ -0.02137124,  0.40618608],
       [ -0.84192557,  0.53058695],
       ...,
       [ -0.94355873,  0.3278936 ],
       [  0.79419406,  0.60777171]])
```

Pengelompokan ini direpresentasikan dalam plot sebelah kiri Gambar 9-14. cukup banyak anomali yang teridentifikasi, ditambah tujuh cluster berbeda. jika kita memperluas lingkungan setiap instance dengan meningkatkan `eps` ke 0.2, kita mendapatkan pengelompokan di sebelah kanan, yang terlihat sempurna. Mari kita lanjutkan dengan model yang ada di slide berikut.



Kelas DBSCAN tidak memiliki `predict()` metode, meskipun memiliki metode `fit_predict()`. Dengan kata lain, dia tidak dapat memprediksi cluster mana yang dimiliki instance baru. Keputusan implementasi ini dibuat karena klasifikasi algoritma yang berbeda bisa lebih baik untuk tugas yang berbeda, jadi pengguna dapat memilih mana yang akan digunakan dan penerapannya juga mudah.

Contohnya, mari kita berlatih KNeighborsClassifier:

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=50)
knn.fit(dbscan.components_, dbscan.labels_[dbscan.core_sample_indices_])
```

Dengan beberapa instance baru, sekarang kita dapat memprediksi kemungkinan besar cluster tersebut dimiliki oleh dan bahkan memperkirakan probabilitas untuk setiap cluster:

```
>>> X_new = np.array([[ -0.5,  0], [ 0,  0.5], [ 1, -0.1], [ 2,  1]])
>>> knn.predict(X_new)
array([1, 0, 1, 0])
>>> knn.predict_proba(X_new)
array([[0.18, 0.82],
       [1.  , 0.  ],
       [0.12, 0.88],
       [1.  , 0.  ]])
```

Perhatikan bahwa kita bisa memprediksi cluster yang dimiliki oleh instance baru ini, tetapi kita dapat juga memilih untuk melatih semua instance, atau semua kecuali yang anomali, pilihan ini tergantung pada tujuan akhir.

Batas keputusan terdapat pada Gambar 9-15 (salib mewakili empat instance di X_{new}). Perhatikan bahwa karena tidak ada anomali dalam set pelatihan, pengklasifikasi selalu memilih sebuah cluster, bahkan ketika cluster itu jauh. cukup mudah untuk memperkenalkan jarak maksimum dan dalam kasus ini dua instance yang jauh dari kedua cluster diklasifikasikan sebagai anomali.

Untuk melakukan ini, gunakan metode `kneighbors()` dari `KNeighborsClassifier`. Diberikan satu set instance, instance mengembalikan jarak dan indeks dari k terdekat dalam set pelatihan (dua matriks, masing-masing dengan kolom k):

```
>>> y_dist, y_pred_idx = knn.kneighbors(X_new, n_neighbors=1)
>>> y_pred = dbscan.labels_[dbscan.core_sample_indices_][y_pred_idx]
>>> y_pred[y_dist > 0.2] = -1
>>> y_pred.ravel()
array([-1,  0,  1, -1])
```

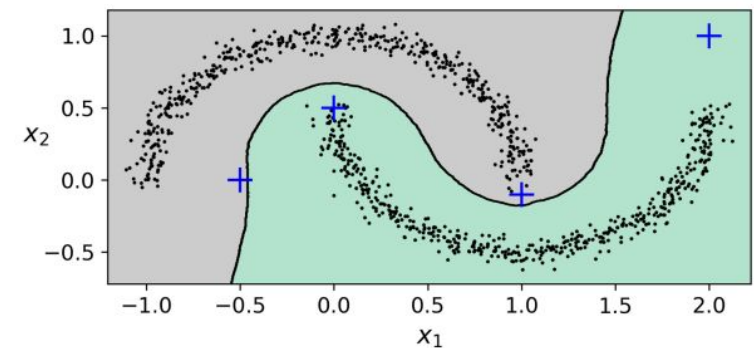


Figure 9-15. Decision boundary between two clusters

Singkatnya, DBSCAN adalah algoritma yang sangat sederhana namun kuat yang mampu mengidentifikasi sejumlah cluster dalam bentuk apa pun. Algoritma ini kuat untuk outlier, dan hanya memiliki dua hyperparameters (eps dan min_samples). Jika kepadatan bervariasi secara signifikan di seluruh cluster, namun, tidak mungkin untuk algoritma ini untuk menangkap semua cluster dengan benar. Kompleksitas komputasinya kira-kira $O(m \log m)$, membuatnya cukup dekat dengan linier sehubungan dengan jumlah instance, tetapi implementasi Scikit-Learn dapat membutuhkan hingga $O(m^2)$ memori jika eps-nya besar.

Other Clustering Algorithms

Scikit-Learn mengimplementasikan beberapa algoritme pengelompokan lagi yang mungkin Anda harus tau. Kita tidak dapat membahas semuanya secara detail di sini, tetapi ini adalah gambaran singkat beberapa algoritme tersebut:

- BIRCH

BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) algoritma ini dirancang khusus untuk dataset yang sangat besar, dan bisa lebih cepat daripada batch K-Means, dengan hasil yang serupa, asalkan jika jumlah fiturnya tidak terlalu besar (<20). Selama pelatihan, birch membangun struktur pohon yang berisi informasi yang cukup untuk dengan cepat menetapkan setiap instance baru ke sebuah cluster, tanpa harus menyimpan semua instance di pohon, pendekatan ini memungkinkannya untuk menggunakan memori kecil saat menangani kumpulan data yang sangat besar.

- Agglomerative clustering (Pengelompokan aglomeratif)

Hirarki cluster dibangun dari bawah ke atas. pada setiap iterasi, pengelompokan aglomeratif menghubungkan pasangan klaster terdekat (dimulai dengan instance individu). Jika Anda menggambar pohon dengan cabang untuk setiap pasang cluster yang bergabung, Anda akan mendapatkan pohon biner cluster, di mana daun adalah instance individu.

Pendekatan ini baik untuk sejumlah besar instance atau cluster agglomerative clustering bisa menangkap berbagai bentuk cluster, menghasilkan pohon cluster yang fleksibel dan informatif daripada memaksa untuk memilih skala cluster tertentu, dan dapat digunakan dengan jarak berpasangan apapun.

Agglomerative clustering dapat menskala dengan baik sejumlah besar instance jika Anda menyediakan matriks konektivitas, yang dimana matriks $m \times m$ menunjukkan pasangan instance yang bertetangga (misalnya, returned by `sklearn.neighbors.kneighbors_graph()`). tanpa matriks konektivitas, algoritme tidak menskalakan dengan baik untuk kumpulan data yang besar.

- Mean-Shift

Algoritma ini dimulai dengan menempatkan lingkaran di tengah setiap instance; kemudian untuk setiap lingkaran dihitung rata-rata dari semua instance yang terletak di dalamnya, dan menggeser lingkaran sehingga berpusat pada mean. Selanjutnya, ia mengulangi langkah pergeseran sampai semua lingkaran berhenti bergerak (sampai masing-masing berpusat pada mean setiap instance di dalamnya). Mean-Shift menggeser lingkaran ke arah kepadatan yang lebih tinggi, sampai masing-masing menemukan kepadatan lokal maksimum. akhirnya, semua instance yang lingkarannya telah menetap di tempat yang sama (atau cukup dekat) ditugaskan ke cluster yang sama.

Mean-Shift memiliki beberapa fitur yang sama dengan DBSCAN, seperti bagaimana ia dapat menemukan berapapun jumlah cluster dalam bentuk apa pun, memiliki sangat sedikit hyperparameter (hanya satu — jari-jari lingkaran, yang disebut bandwidth), dan itu bergantung pada estimasi kepadatan lokal. tetapi tidak seperti DBSCAN, Mean-Shift cenderung memotong cluster menjadi potongan-potongan ketika mereka memiliki variasi kepadatan internal. Sayangnya, kompleksitas komputasinya adalah $O(m^2)$ sehingga tidak cocok untuk dataset yang besar.

- Affinity propagation

Algoritma ini menggunakan sistem voting, di mana instance memilih instance yang serupa untuk menjadi perwakilan mereka, dan setelah konvergen algoritma, masing-masing perwakilan dan pemilihnya membentuk sebuah cluster. affinity propagation dapat mendeteksi berapapun cluster dengan ukuran berbeda. sayangnya, algoritma ini memiliki kompleksitas komputasi $O(m^2)$ sehingga tidak cocok untuk dataset yang besar.

- Spectral clustering

Algoritma ini mengambil matriks serupa antara instance dan membuat embedding berdimensi rendah darinya, kemudian menggunakan algoritma pengelompokan lain di ruang dimensi rendah (implementasi Scikit-Learn menggunakan K-Means) Pengelompokan spektral dapat menangkap struktur cluster yang kompleks, dan dapat juga digunakan untuk memotong grafik (misalnya, untuk mengidentifikasi kelompok teman di dalam sebuah jaringan sosial). spectral clustering tidak mengskalakan dengan baik untuk sejumlah besar instance, dan tidak berfungsi dengan baik ketika cluster memiliki ukuran yang sangat berbeda.

Chapter 9: Bayesian Gaussian Mixture Models, Other Algorithms for Anomaly and Novelty Detection

Bayesian Gaussian Mixture Models

Bayesian Gaussian Mixture merupakan sebuah kelas yang secara otomatis mencari jumlah cluster yang optimal, karena mampu memberikan bobot yang sama (atau mendekati) ke nol untuk cluster yang tidak perlu.

Langkah menggunakan kelas ini :

- > Tetapkan jumlah mengelompokkan n komponen ke nilai yang telah diyakini lebih besar dari jumlah optimal cluster (ini mengamsumsikan beberapa pengetahuan tentang masalah yang dihadapi),
- > Kemudian, algoritma akan menghilangkan cluster yang tidak perlu secara otomatis.

Contoh :

```
>>> from sklearn.mixture import BayesianGaussianMixture  
>>> bgm = BayesianGaussianMixture(n_components=10, n_init=10)  
>>> bgm.fit(X)  
>>> np.round(bgm.weights_, 2)  
array([0.4 , 0.21, 0.4 , 0. , 0. , 0. , 0. , 0. , 0. , 0. ])
```

Penjelasan :

Dari contoh diatas, Algoritma secara otomatis mendeteksi bahwa hanya tiga cluster yang dibutuhkan.

Contoh Lain:

Contoh model disamping, merupakan parameter cluster (termasuk bobot, rata-rata, dan matriks kovarians) tidak diperlakukan sebagai parameter model tetap lagi, tetapi sebagai variabel acak.

Penjelasan Gambar di samping :

- Jadi z sekarang termasuk parameter cluster dan cluster tugas.

- Distribusi Beta(Φ) biasanya digunakan untuk memodelkan variabel acak yang nilai terletak dalam kisaran tetap. Dalam hal ini, kisarannya adalah dari 0 hingga 1.

The Stick-Breaking Process (SBP) paling baik dijelaskan melalui sebuah contoh:

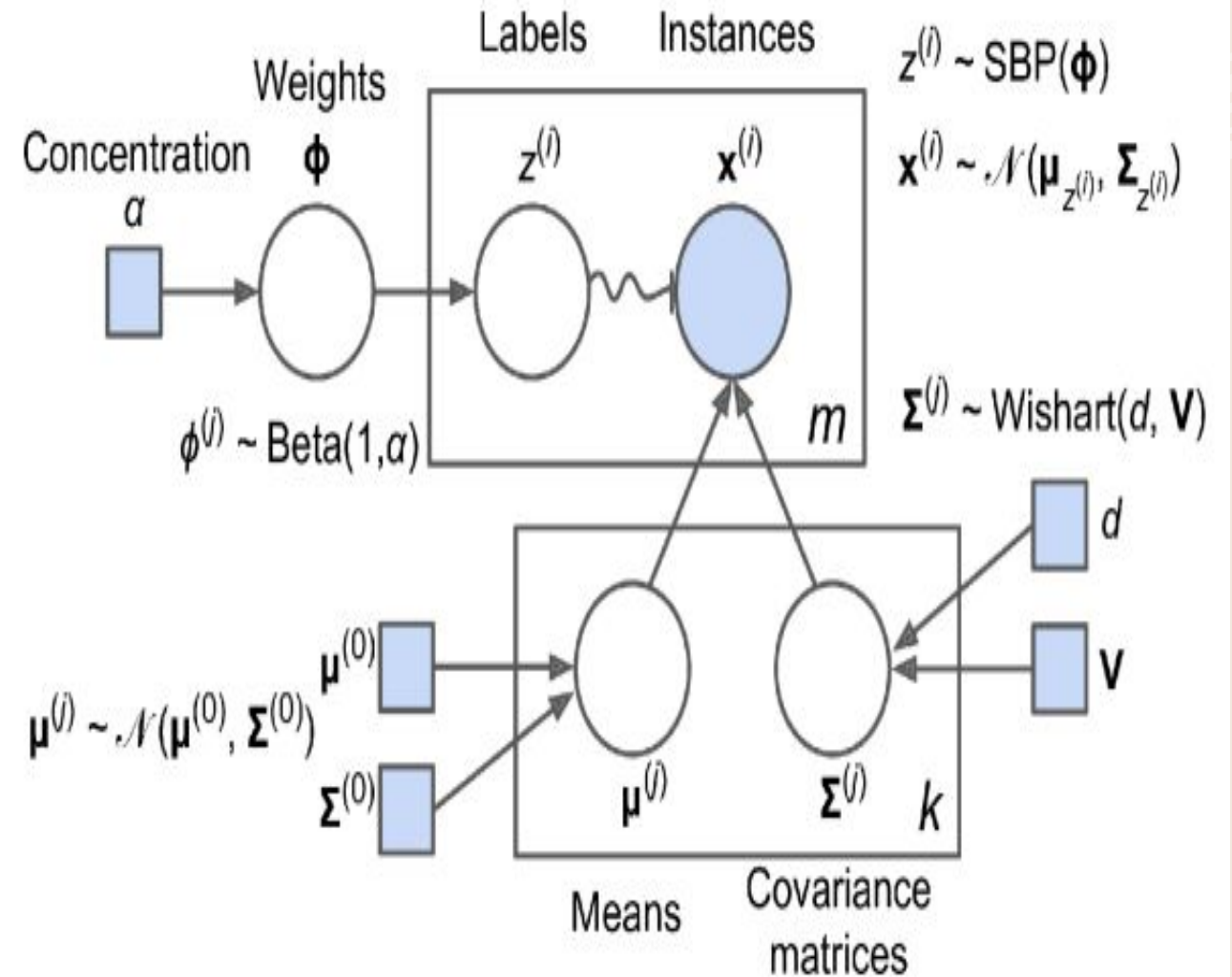


Figure 9-22. Bayesian Gaussian mixture model

misalkan $= [0.3, 0.6, 0.5, \dots]$,

- maka 30% dari instance akan ditetapkan ke cluster 0,
- maka 60% dari instance yang tersisa akan ditugaskan ke cluster 1,
- maka 50% dari instance yang tersisa akan ditetapkan ke cluster 2,

Proses di atas adalah model yang baik untuk kumpulan data di mana instance baru berada lebih mungkin untuk bergabung dengan kelompok besar daripada kelompok kecil (misalnya, orang lebih cenderung pindah ke kota-kota besar). Jika konsentrasi tinggi, maka nilai kemungkinan akan mendekati 0, dan SBP menghasilkan banyak cluster. Sebaliknya, jika konsentrasinya rendah, maka nilai kemungkinan akan mendekati ke 1, dan akan ada beberapa cluster. Akhirnya, distribusi Wishart adalah digunakan untuk sampel matriks kovarians: parameter d dan V mengontrol distribusi bentuk cluster.

Catatan Penting :

- > Semakin banyak data yang dimiliki, maka semakin sedikit prioritas. Sebenarnya, untuk memplot diagram dengan perbedaan besar seperti itu, maka harus menggunakan prior yang sangat kuat dan sedikit data.
- > Pengetahuan sebelumnya tentang variabel laten z dapat dikodekan dalam a distribusi probabilitas $p(z)$ disebut prior. Misalnya, kita mungkin memiliki keyakinan sebelumnya bahwa kelompok cenderung sedikit (konsentrasi rendah), atau sebaliknya, bahwa mereka cenderung berlimpah (konsentrasi tinggi).

Persamaan 9 - 2 : Teorema Bayes

$$p(\mathbf{z}|\mathbf{X}) = \text{posterior} = \frac{\text{likelihood} \times \text{prior}}{\text{evidence}} = \frac{p(\mathbf{X}|\mathbf{z}) p(\mathbf{z})}{p(\mathbf{X})}$$

Persamaan teorema Bayes diatas, memberi tahu kita cara memperbarui probabilitas distribusi variabel laten setelah kita mengamati beberapa data $\mathbf{X}$ menghitung distribusi posterior $p(\mathbf{z}|\mathbf{X})$, yang merupakan kondisional peluang $\mathbf{z}$ diberikan $\mathbf{X}$.

Sayangnya, dalam model campuran Gaussian (dan banyak masalah lainnya), penyebut $p(\mathbf{x})$ tidak dapat dipecahkan, karena memerlukan integrasi atas semua nilai-nilai yang mungkin dari $\mathbf{z}$ (Persamaan 9-3), yang memerlukan pertimbangan semua kemungkinan kombinasi parameter cluster dan tugas cluster.

Persamaan 9 - 3 : Bukti $p(X)$ sering kali sulit dipecahkan

$$p(X) = \int p(X|z)p(z)dz$$

Ketangguhan ini adalah salah satu masalah utama dalam statistik Bayesian, dan ada beberapa pendekatan untuk menyelesaikannya. Salah satunya adalah variasi inferensi, yang mengambil keluarga distribusi $q(z; \lambda)$ dengan miliknya sendiri parameter variasi (λ), lalu optimalkan parameter ini ke buat $q(z)$ aproksimasi yang baik untuk $p(z|X)$. Hal ini dicapai dengan menemukan nilai yang meminimalkan divergensi KL dari $q(z)$ ke $p(z|X)$, dicatat $D(q||p)$.

Persamaan 9 - 4 : Divergensi KL dari $q(\mathbf{z})$ ke $p(\mathbf{z}|\mathbf{X})$

$$\begin{aligned} D_{KL}(q \parallel p) &= \mathbb{E}_q \left[\log \frac{q(\mathbf{z})}{p(\mathbf{z} | \mathbf{X})} \right] \\ &= \mathbb{E}_q [\log q(\mathbf{z}) - \log p(\mathbf{z} | \mathbf{X})] \\ &= \mathbb{E}_q \left[\log q(\mathbf{z}) - \log \frac{p(\mathbf{z}, \mathbf{X})}{p(\mathbf{X})} \right] \\ &= \mathbb{E}_q [\log q(\mathbf{z}) - \log p(\mathbf{z}, \mathbf{X}) + \log p(\mathbf{X})] \\ &= \mathbb{E}_q [\log q(\mathbf{z})] - \mathbb{E}_q [\log p(\mathbf{z}, \mathbf{X})] + \mathbb{E}_q [\log p(\mathbf{X})] \\ &= \mathbb{E}_q [\log p(\mathbf{X})] - (\mathbb{E}_q [\log p(\mathbf{z}, \mathbf{X})] - \mathbb{E}_q [\log q(\mathbf{z})]) \\ &= \log p(\mathbf{X}) - \text{ELBO} \\ \text{where ELBO} &= \mathbb{E}_q [\log p(\mathbf{z}, \mathbf{X})] - \mathbb{E}_q [\log q(\mathbf{z})] \end{aligned}$$

Dalam praktiknya, ada berbagai teknik untuk memaksimalkan ELBO. berarti inferensi variasi lapangan, perlu untuk memilih keluarga dari distribusi $q(\mathbf{z};)$ dan $p(\mathbf{z})$ sebelumnya dengan sangat hati-hati untuk memastikan bahwa persamaan untuk ELBO disederhanakan menjadi bentuk yang dapat dihitung. Sayangnya, tidak ada cara umum untuk melakukan ini.

Catatan Penting :

- Kompleksitas komputasi dari Kelas Bayesian Gaussian Mixture mirip dengan Kelas Gaussian Mixture (tetapi umumnya jauh lebih lambat).
- Lebih sederhana pendekatan untuk memaksimalkan ELBO disebut stokastik kotak hitam inferensi variasional (BBSVI): pada setiap iterasi, beberapa sampel diambil dari q , dan mereka digunakan untuk memperkirakan gradien ELBO dengan sehubungan dengan parameter variasi, yang kemudian digunakan dalam gradien langkah pendakian.
- Pendekatan ini memungkinkan untuk menggunakan inferensi Bayesian dengan model apa pun (asalkan dapat dibedakan), bahkan neural dalam jaringan; menggunakan inferensi Bayesian dengan jaringan saraf dalam disebut Pembelajaran Mendalam Bayesian.

Other Algorithms for Anomaly and Novelty Detection

Scikit-Learn mengimplementasikan beberapa algoritma lain yang didedikasikan untuk **deteksi anomali** atau **deteksi kebaruan**:

1> **PCA** (dan teknik pengurangan dimensi lainnya dengan `inverse_transform()` metode)

- Jika Anda membandingkan kesalahan rekonstruksi dari contoh normal dengan kesalahan rekonstruksi anomali, yang terakhir biasanya akan banyak lebih besar.
- Ini adalah deteksi anomali yang sederhana dan seringkali cukup efisien.

2> Fast-MCD (penentu kovarians minimum)

- Algoritma ini berguna untuk deteksi outlier, khususnya untuk membersihkan kumpulan data.
- Ini mengasumsikan bahwa instance normal (inliers) dihasilkan dari satu Gaussian distribusi (bukan campuran).
- Ini juga mengasumsikan bahwa dataset adalah terkontaminasi dengan outlier yang tidak dihasilkan dari Gaussian ini distribusi.
- Ketika algoritma memperkirakan parameter dari Distribusi Gaussian (yaitu, bentuk amplop elips di sekitar inlier), berhati-hatilah untuk mengabaikan contoh yang paling mungkin outlier.
- Teknik ini memberikan estimasi elips yang lebih baik amplop dan dengan demikian membuat algoritma lebih baik dalam mengidentifikasi outlier.

3> Isolation Forest

- Algoritma ini adalah algoritma yang efisien untuk deteksi outlier, terutama dalam dataset dimensi tinggi.
- Algoritma membangun Hutan Acak di mana setiap Pohon Keputusan ditumbuhkan secara acak: di setiap simpul, ia mengambil fitur secara acak,
- kemudian mengambil nilai ambang acak (antara min dan nilai maks) untuk membagi dataset menjadi dua.
- Anomali biasanya jauh dari contoh lain, jadi rata-rata (di semua Pohon Keputusan) mereka cenderung terisolasi di langkah lebih sedikit daripada contoh normal.

4> LOF (Local Outlier Factor)

- Algoritma ini juga bagus untuk deteksi outlier. Ini membandingkan kepadatan instance di sekitar instance yang diberikan dengan kepadatan di sekitar tetangga.
- Suatu anomali seringkali lebih terisolasi daripada k terdekatnya tetangga.

5> One-class SVM

- Algoritma ini lebih cocok untuk deteksi kebaruan.
- Pengklasifikasi SVM kernelized memisahkan dua kelas dengan yang pertama (secara implisit) memetakan semua instance ke ruang dimensi tinggi, lalu memisahkan dua kelas menggunakan pengklasifikasi SVM linier dalam ruang dimensi tinggi.
- Jika instance baru tidak termasuk dalam wilayah ini, itu adalah sebuah anomali.
- Ada beberapa hyperparameter untuk diubah: yang biasa untuk SVM kernel, ditambah hyperparameter margin yang sesuai dengan kemungkinan contoh baru yang keliru dianggap sebagai novel padahal sebenarnya biasa saja.
- Perubahan berfungsi dengan baik, terutama dengan kumpulan data dimensi tinggi, tetapi seperti semua SVM, ini tidak berskala besar kumpulan data.



UNIVERSITAS
KRISTEN
MARANATHA